

A Comprehensive Blueprint for Building Functional AI Agent Frameworks

I. Introduction to AI Agent Frameworks

A. Defining AI Agents and Frameworks

Artificial Intelligence (AI) agents represent a significant evolution beyond simple Large Language Model (LLM) interactions. They are autonomous programs designed to leverage LLMs as reasoning engines, enabling them to perceive their environment, make decisions, plan actions, and utilize external tools to achieve specific goals.¹ Unlike basic LLM calls that primarily generate text, agents can perform complex tasks, interact with external systems, and operate with a degree of autonomy.³

The development of these sophisticated agents is facilitated by AI Agent Frameworks. These frameworks are libraries, toolkits, or SDKs that provide structure, abstractions, and pre-built components to simplify and accelerate the process of building, deploying, and managing AI agents.¹ They address inherent LLM limitations, such as imperfect memory, difficulty adhering strictly to logic, and inability to interact directly with external environments or perform computations.⁶ Frameworks offer solutions for integrating tools, managing conversational memory (state), accessing external knowledge bases, and orchestrating complex workflows involving reasoning and planning.¹

B. Overview of the Agent Framework Landscape

The landscape of AI agent frameworks is diverse and rapidly evolving, reflecting different design philosophies and priorities. Key frameworks include:

1. **Agno:** A lightweight Python library emphasizing performance, simplicity ("pure Python"), native multi-modal support, and model agnosticism.¹ It positions itself against complex abstractions like graphs or chains.⁷
2. **LangChain:** A widely adopted framework known for its extensive ecosystem of integrations, composable components, and the LangChain Expression Language (LCEL) for declarative chain building.¹² It provides a standard interface for models, embeddings, vector stores, etc..¹³
3. **LangGraph:** An extension of LangChain designed for building stateful, multi-actor applications. It uses a graph-based approach (nodes and edges) for explicit control over complex, potentially cyclical agent workflows.¹¹ It is recommended over the legacy LangChain AgentExecutor for building agents.²⁷
4. **OpenAI Agents SDK:** A Python-first framework from OpenAI for building single and multi-agent workflows, featuring concepts like Agents, Tools (including

Function Tools), Handoffs for delegation, and Guardrails for safety.²⁸ Evolved from the experimental Swarm project.³⁷

5. **Google Agent Development Kit (ADK):** An open-source, code-first Python framework emphasizing flexibility, control, and tight integration with the Google Cloud ecosystem (Vertex AI, Gemini models).⁵ Supports various agent types (LLM, Workflow - Sequential, Parallel, Loop) and multi-agent systems.⁵
6. **Other Frameworks:** The ecosystem also includes frameworks like AutoGen (Microsoft, multi-agent conversations)¹¹, CrewAI (role-playing agents)¹⁹, Phidata (now Agno)¹⁰, and others.³⁹

These frameworks exist on a continuum, ranging from simpler, deterministic chains (predefined workflows) to single-agent systems capable of dynamic tool use, and finally to complex multi-agent architectures involving coordination and collaboration.⁴⁸ The evolution from early LangChain concepts towards more structured and controllable frameworks like LangGraph, OpenAI SDK, Google ADK, and the performance-focused Agno signifies a maturation of the field. This progression addresses the growing need for reliability, observability, manageability, and performance as agentic systems move from prototypes towards production deployments.¹

C. Core Value Proposition

Developers utilize agent frameworks primarily because they abstract away significant low-level complexity involved in building agentic systems. Instead of manually coding the logic for LLM interaction, prompt management, tool parsing, memory handling, and workflow orchestration, frameworks provide reusable components and standardized interfaces. This accelerates development, allows for easier integration of diverse tools and models, facilitates the implementation of advanced features like long-term memory and Retrieval-Augmented Generation (RAG), and often includes mechanisms for evaluation, debugging, and deployment.⁵

D. Report Roadmap

This report provides a comprehensive blueprint for designing and building a functional AI agent framework. It will delve into:

- **Core Concepts:** The fundamental building blocks common to most agents (Models, Instructions, Tools, Memory, Knowledge, Reasoning).
- **Architectural Patterns:** Comparing different framework philosophies and design choices (Lightweight, Composable, Graph-Based, SDKs) and exploring single vs. multi-agent systems.
- **Building Blocks:** A step-by-step guide covering environment setup, agent

definition, and integration of core components.

- **Advanced Features:** Discussing multi-agent orchestration, performance optimization, observability, security, and error handling.
- **Evaluation and Deployment:** Strategies for testing agent performance and options for deploying frameworks.
- **Conclusion:** Summarizing the key considerations and future directions in agent framework development.

II. Core Concepts of AI Agents

Understanding the fundamental components that constitute an AI agent is crucial before designing a framework. While implementations vary, several core concepts are nearly universal across the landscape.

A. The Agent Core: Model + Instructions

At the heart of every agent lies the interaction between a Large Language Model (LLM) and the instructions guiding its behavior.

- **Model:** The LLM serves as the agent's cognitive engine, responsible for understanding input, reasoning, planning, and generating responses or actions.¹ Frameworks often aim for model agnosticism, allowing developers to plug in various LLMs from different providers (e.g., OpenAI's GPT series, Anthropic's Claude, Google's Gemini, Meta's Llama, Mistral models, Groq LPUs).¹ Examples include using `agno.models.openai.OpenAIChat`¹, `agno.models.anthropic.Claude`¹, `agno.models.groq.Groq`⁵¹, or LangChain's model wrappers.¹² However, some frameworks like Google ADK are optimized for specific ecosystems, offering tighter integration with models like Gemini.⁵ Configuration typically involves specifying the model ID (e.g., "gpt-4o", "claude-3-7-sonnet-latest", "llama-3.1-70b-versatile") and standard LLM parameters like temperature or max tokens.³¹
- **Instructions/Prompting:** These are the explicit guidelines defining the agent's persona, goal, capabilities, constraints, and operational procedures.¹ Effective prompting is critical for reliable agent behavior.² Frameworks provide mechanisms to structure these instructions, often through system messages or dedicated parameters (description, instructions, goal, role).⁵⁰ Prompt templates (like LangChain's `ChatPromptTemplate`⁵³) allow for dynamic insertion of context, such as user input, chat history, or tool descriptions. Some frameworks offer helpers like adding timestamps to instructions for context.⁵⁷ Crafting clear, unambiguous instructions, potentially derived from existing documentation or procedures, is a key aspect of agent design.³⁰

B. Tools: Extending Capabilities

LLMs alone cannot interact with the external world or perform specialized computations. Tools bridge this gap, providing agents with specific functionalities.

- **Definition:** Tools are essentially functions or APIs that the agent's LLM can decide to call during its execution loop to gather information, perform actions, or leverage external services.¹
- **Types:** Frameworks typically support various tool types:
 - **Pre-built Tools:** Common functionalities packaged for ease of use, such as web search (DuckDuckGoTools¹, TavilySearchResults¹⁸, OpenAI WebSearchTool²⁸), financial data retrieval (YFinanceTools¹), file system interaction¹¹, code execution (PythonREPLTool⁵⁸, ADK Code Exec⁵), database interaction, or API calls.
 - **Custom Tools:** Developers can wrap their own Python functions to create bespoke tools tailored to specific application needs.⁵ Frameworks often provide decorators (@function_tool in OpenAI SDK²⁸) or helper functions (tool in LangChain¹⁴) to simplify this process.
 - **Schema-Defined Tools:** Tools can be defined using standards like OpenAPI specifications, allowing agents to interact with existing APIs.⁵
 - **Protocol-Based Tools:** Emerging standards like the Model Context Protocol (MCP) allow tools to be exposed by separate servers and discovered/used by agents via a standardized protocol.⁴⁵
- **Integration:** Tools are typically provided to the agent during initialization via a tools parameter, which accepts a list of tool instances or functions.³³ Crucially, each tool requires a clear and accurate description detailing its purpose and parameters. The LLM relies heavily on these descriptions to determine *when* to use a tool and *how* to invoke it correctly.² Defining input schemas (e.g., using Pydantic⁵⁰ or Zod⁶¹) for tools further aids the LLM in generating correct invocations and allows for input validation. Frameworks may offer options to display tool calls during execution for debugging (show_tool_calls in Agno⁵⁰) or control tool usage (tool_choice in OpenAI⁶⁵).

C. Memory: Maintaining Context

To engage in coherent, multi-turn conversations or carry context across complex tasks, agents require memory.

- **Purpose:** Memory mechanisms store and retrieve information from past interactions, allowing agents to maintain context, recall user preferences, and avoid redundant questions.¹

- **Types & Implementations:** Frameworks offer diverse memory strategies, often distinguishing between short-term (within a single session) and long-term (across sessions) memory ²¹:
 - **Buffer Memory:** Stores a raw log of conversation messages (e.g., LangChain ConversationBufferMemory ⁷⁸, Agno Chat History ⁸). Simple but can exceed context limits.
 - **Windowed Buffer Memory:** Keeps only the most recent 'k' interactions (e.g., LangChain ConversationBufferWindowMemory ⁷⁸).
 - **Token Buffer Memory:** Limits the buffer size based on token count, dropping older messages when the limit is reached (e.g., LangChain ConversationTokenBufferMemory ⁷⁸).
 - **Summary Memory:** Uses an LLM to progressively summarize the conversation, keeping the context concise (e.g., LangChain ConversationSummaryMemory ⁷⁸, Agno Session Summaries ⁸). Trades detail for brevity.
 - **Vector Store Memory:** Embeds conversation turns and stores them in a vector database, allowing retrieval of semantically relevant past interactions (e.g., LangChain VectorStoreRetrieverMemory ⁷⁸).
 - **Entity Memory:** Extracts key entities (people, places, concepts) and summarizes information related to them (e.g., LangChain ConversationEntityMemory ⁷⁸).
 - **Framework-Specific Implementations:**
 - **Agno:** Offers pluggable Memory and AgentStorage drivers, supporting backends like SQLite (SqliteAgentStorage ⁵⁴). Distinguishes Chat History, User Memories (persistent key-value), and Session Summaries.⁸
 - **Google ADK:** Separates concepts clearly: Session tracks events in the current interaction, State holds temporary data within that session, and MemoryService manages searchable, long-term knowledge potentially spanning multiple sessions (using InMemory or Vertex AI RAG backends).⁴¹ SessionService manages the lifecycle of Session objects.⁸⁷
 - **LangGraph:** Manages state explicitly within the graph structure, allowing nodes to update a shared state object.¹⁹ Persistence can be added via savers like SqliteSaver.²⁴
- **Integration:** Memory components are typically passed to the agent or execution runtime during initialization (e.g., Agno storage param ⁵⁷, LangChain memory param ⁹², ADK SessionService in Runner ⁸⁷). Flags like add_history_to_messages control whether the framework automatically injects memory content into prompts.⁵⁵ The sophistication of memory management reflects the challenge of providing relevant context within LLM limitations, moving beyond simple buffers

towards more intelligent summarization and retrieval techniques.

D. Knowledge: Accessing External Information (Agentic RAG)

Agents often need access to specific, external knowledge bases (documents, databases, APIs) to answer questions accurately or perform domain-specific tasks. Retrieval-Augmented Generation (RAG) is the primary mechanism for this.

- **Purpose:** To ground agent responses in factual, up-to-date, or proprietary information not present in the LLM's training data.¹
- **Mechanism - Agentic RAG:** A significant evolution from traditional RAG is "Agentic RAG".⁵⁰ In traditional RAG, the retrieval step (querying a knowledge base) usually happens once, upfront, based on the initial user query, with the retrieved context passed to the LLM. In Agentic RAG, the agent *itself* controls the retrieval process. It can decide *during* its reasoning loop whether it needs information, formulate specific queries for its knowledge base, and potentially perform multiple retrieval steps as its understanding evolves.¹ This grants the agent more autonomy and flexibility in information gathering. Agno implements Agentic RAG by default.¹
- **Components:** Implementing RAG involves several standard components:
 - **Document Loaders:** Ingest data from various sources (PDFs, URLs, databases, APIs) into a standard Document format (text content + metadata).⁹² Examples: Agno UrlKnowledge, PdfUrlKnowledgeBase¹; LangChain loaders.⁹⁴
 - **Text Splitters:** Break large documents into smaller, semantically meaningful chunks suitable for embedding and retrieval.⁹²
 - **Embedding Models:** Convert text chunks into numerical vector representations.¹ Examples: OpenAIEmbedder, SentenceTransformerEmbedder, GeminiEmbedder.⁵⁴
 - **Vector Stores:** Databases optimized for storing and efficiently searching high-dimensional vectors based on similarity.⁵³ Examples: LanceDb¹, PgVector⁶⁰, Milvus⁸⁴, Weaviate⁹⁹, FAISS.¹⁰⁰
 - **Retrievers:** The component responsible for taking a query string and returning relevant documents, often interacting with a vector store.⁵³
- **Implementation:** Frameworks provide abstractions to manage these components. Agno uses AgentKnowledge objects, configured with a vector_db and embedder.⁵⁴ LangChain offers various RAG chains and retriever types.¹⁰¹ OpenAI provides a built-in file_search tool leveraging vector stores.²⁸ Google ADK integrates RAG via its MemoryService connected to Vertex AI RAG.⁸⁶ Agents are typically configured with a knowledge base object (e.g., Agno knowledge

parameter⁵⁰) and potentially a dedicated tool for searching it (Agno search_knowledge⁵⁵).

E. Reasoning: Enabling Thought Processes

A key differentiator for agents is their ability to reason – to break down problems, plan steps, and think logically towards a solution.

- **Purpose:** To move beyond simple input-output mapping and enable agents to tackle multi-step, complex tasks requiring planning and analysis.¹
- **Techniques:**
 - **Implicit Reasoning:** Relying on the inherent reasoning capabilities of advanced LLMs, guided by carefully crafted instructions.
 - **Chain-of-Thought (CoT):** Prompting the LLM to "think step-by-step" before providing the final answer, making its reasoning process more explicit and often improving accuracy.
 - **ReAct (Reason + Act):** A specific agent loop pattern where the LLM explicitly generates a 'Thought' (reasoning step), then an 'Action' (tool to use or final answer), receives an 'Observation' (tool result), and iterates.⁵⁸ LangChain provides specific ReAct agent types and prompts.⁵⁸
 - **Dedicated Reasoning Components:** Some frameworks offer specific tools or models for reasoning. Agno treats reasoning as a "first-class citizen"¹ and provides ReasoningTools¹ or allows specifying a separate reasoning_model or reasoning_agent.¹
- **Framework Support:** The level of explicit support for reasoning varies. Agno highlights it¹, LangChain offers ReAct agents⁶², while other frameworks might rely more on the LLM's inherent abilities combined with structured prompting or tool use. Configuration might involve enabling reasoning flags or setting reasoning step limits.⁵⁷

These core concepts – Model, Instructions, Tools, Memory, Knowledge, and Reasoning – form the essential pillars upon which AI agent frameworks are built. While universally relevant, their specific implementations and the emphasis placed on each vary significantly across different frameworks, reflecting diverse architectural philosophies and target use cases. The shift towards Agentic RAG and more sophisticated memory management highlights the ongoing effort to overcome LLM limitations and build more capable, context-aware, and autonomous agents.

III. Architectural Patterns & Design Choices

Building an AI agent framework involves making fundamental architectural decisions

that impact performance, flexibility, scalability, and the developer experience. Understanding the different patterns and philosophies is key to selecting or designing an appropriate framework.

A. Framework Architectures

The current landscape showcases several distinct architectural approaches:

1. **Lightweight Libraries (e.g., Agno):** This approach prioritizes simplicity, performance, and minimal abstraction layers.¹ Agno, for example, advocates for "pure Python" without "convoluted graphs, chains, or unnecessary patterns".⁷ Key characteristics often include:
 - **Performance Focus:** Emphasis on low agent instantiation times (Agno claims ~2-3 μ s) and minimal memory footprints (Agno claims ~3.75-5KiB), often benchmarked against other frameworks like LangGraph.¹ This focus is crucial for scalability, especially in multi-agent systems where many agents might be active concurrently.
 - **Model Agnosticism:** Designed to work with a wide range of LLM providers without vendor lock-in.¹
 - **Native Multi-modality:** Built from the ground up to handle text, image, audio, and video inputs/outputs.¹
 - **Potential Trade-off:** May require developers to implement more orchestration logic manually compared to frameworks with higher-level abstractions for complex stateful workflows.
2. **Composable Frameworks (e.g., LangChain):** These frameworks emphasize modularity and a rich ecosystem of integrations, allowing developers to connect various components like building blocks.¹² LangChain exemplifies this with:
 - **Core Abstractions:** Standardized interfaces (Runnable) for models, prompts, retrievers, etc., promoting interoperability.¹⁴
 - **LangChain Expression Language (LCEL):** A declarative syntax using the pipe operator (|) to compose Runnables into chains.⁶³ LCEL provides benefits like automatic streaming, asynchronous support, parallel execution optimization, and integration with observability tools like LangSmith.⁵³
 - **Ecosystem:** A vast collection of community and partner integrations for models, tools, and data sources.¹²
 - **Potential Trade-off:** The level of abstraction, while simplifying composition, can sometimes make debugging complex chains more challenging.¹¹³ The original AgentExecutor concept proved less flexible for advanced agents, leading to the development of LangGraph.²⁷
3. **Graph-Based Orchestration (e.g., LangGraph):** This architecture explicitly

models workflows as graphs, where nodes represent computational steps (e.g., LLM calls, tool executions) and edges represent the flow of control and data.¹¹

Key features include:

- **Explicit State Management:** A central state object is passed between nodes, allowing for robust management of information throughout the workflow.²⁰ State updates can overwrite or add to existing values.²²
 - **Cyclical Workflows:** Naturally supports loops and conditional branching, essential for iterative reasoning, error recovery, and human-in-the-loop processes.²¹
 - **Control and Flexibility:** Offers fine-grained control over the agent's execution flow, making it suitable for complex, multi-step, and multi-agent applications.²¹
 - **Potential Trade-off:** Can involve a steeper learning curve compared to simpler chaining mechanisms due to the need to define graph structure, nodes, edges, and state transitions explicitly.
4. **SDKs (e.g., OpenAI Agents SDK, Google ADK):** These Software Development Kits provide a more structured, often opinionated, set of tools and primitives specifically designed for building agents within a particular ecosystem or following a specific methodology.⁵
- **Defined Primitives:** Offer core concepts like Agent, Tool, Runner, and often specialized features like Handoffs (OpenAI ²⁸) or Workflow Agents (Google ADK ⁵).
 - **Python-First Approach:** Often designed for ease of use within Python, leveraging decorators and standard language features.⁵
 - **Ecosystem Integration:** May offer tighter integration with the provider's models, services, and deployment platforms (e.g., ADK with Gemini/Vertex AI ⁵, OpenAI SDK with OpenAI models/tracing ³³).
 - **Built-in Features:** Often include integrated tracing/observability and safety features like Guardrails (OpenAI SDK ²⁸).
 - **Potential Trade-off:** Might be less model-agnostic or flexible outside their intended ecosystem compared to more general-purpose libraries.

The choice between these architectures involves balancing factors like desired performance, development speed, need for control over execution flow, complexity of the target application, and integration requirements with specific platforms or models. The trend towards graph-based systems and structured SDKs suggests a move towards more robust and manageable solutions for building production-grade agentic applications.

Table 1: Comparison of AI Agent Framework Architectures

Feature	Agno	LangChain/ LCEL	LangGraph	OpenAI Agents SDK	Google ADK
Core Philosophy	Lightweight, Performance, Simplicity ("Pure Python"), Multi-modal	Composability, Ecosystem, Abstraction (Runnables), Declarative Chains	Explicit State Management, Graph-based Control Flow, Complex/Cyclical Workflows	Python-first, Structured Workflows, Multi-agent Orchestration (Handoffs), Safety	Code-first Control, Modularity, Google Cloud/Gemini Integration, Evaluation
Key Abstractions	Agent, Tool, Memory, AgentKnowledge, AgentStorage	Runnable, ChatModel, PromptTemplate, Retriever, Tool, OutputParser	StateGraph, Nodes, Edges, State Object	Agent, Tool (@function_tool), Handoff, Guardrail, Runner	Agent (LLM, Workflow), Tool, Runner, Session, State, MemoryService
State Management	Pluggable AgentStorage drivers (e.g., SQLite), Session/User Memory	Basic Memory classes (ConversationBufferMemory, etc.), LCEL state passing	Explicit graph state object, Reducers (add/overwrite), Persistence via Savers	Implicit within Runner loop context	Explicit Session, State, MemoryService (InMemory, Vertex AI RAG)
Multi-Agent Support	Team (route, collaborate, coordinate)	Possible via composition, but LangGraph preferred	Designed for multi-actor systems, explicit graph structure	Handoffs for delegation	Hierarchical systems, Delegation via LLM routing based on descriptions, A2A protocol support ⁸⁸

Performance Focus	Very High (low latency/memory) ¹	Moderate (LCEL optimizations)	Moderate (focus on control)	Moderate	Moderate
Ecosystem Integration	Model Agnostic (23+ providers), Vector DBs (20+) ¹	Very High (700+ integrations via core, community, partner packages) ¹⁴	Leverages LangChain ecosystem	Primarily OpenAI models, Extensible tracing	Tight Google Cloud/Gemini integration, Supports MCP/LangChain tools ⁵
Strengths	Speed, Efficiency, Simplicity, Native Multi-modality	Ease of Composition, Vast Ecosystem, Flexibility	Control over complex flows, Statefulness, Cycles, Human-in-the-loop	Structured approach, Clear primitives (Handoffs), Built-in tracing/guardrails	Fine-grained control, Testability, Scalability via Vertex AI, Built-in evaluation
Weaknesses	Newer ecosystem, May require more manual orchestration	Abstraction complexity, Legacy AgentExecution or limitations	Steeper learning curve	Primarily OpenAI focused, Newer SDK	Primarily Google focused, Can be verbose

B. Single-Agent vs. Multi-Agent Systems

A fundamental design decision is whether to build a single, potentially complex agent or decompose the problem into a system of multiple, collaborating agents.

- **Single-Agent Systems:** A single agent, equipped with multiple tools and potentially complex internal logic (like ReAct or iterative refinement loops), handles the entire task. ⁴⁸
 - **Suitability:** Effective for moderately complex tasks confined to a specific domain where a single reasoning engine can manage the required tools and context. ⁴⁸ Often considered a good default starting point. ⁴⁸
 - **Example:** A research assistant agent that can search the web, read documents, and synthesize a report.
 - **Limitations:** Can hit a "complexity threshold" where the prompt becomes too long and convoluted, the number of tools becomes unmanageable for the

LLM, or the required reasoning spans too many disparate areas, leading to reduced performance and reliability.⁶¹ Debugging can become difficult.

- **Multi-Agent Systems:** The task is broken down, and multiple specialized agents collaborate to achieve the overall goal.¹
 - **Suitability:** Necessary for large, complex, or cross-functional problems where different expertise or processing steps are required.⁴⁸ Improves modularity, specialization, reusability, and maintainability.⁵ Allows each agent to have a simpler, more focused prompt and toolset.⁶¹
 - **Orchestration Patterns:** How these agents interact is key:
 - **Routing/Delegation:** A central agent (supervisor, router, triage) directs tasks to appropriate specialist agents based on input or context. Examples: Agno route mode ¹, OpenAI Triage Agent using handoffs ²⁸, ADK LLM-driven routing based on agent descriptions.⁴⁵
 - **Collaboration:** Agents work together, potentially sharing state or passing information back and forth. Example: Agno collaborate mode.¹
 - **Coordination:** A central agent manages the workflow, assigns tasks, and integrates results from worker agents. Example: Agno coordinate mode ¹ (e.g., Web Agent + Finance Agent team ¹).
 - **Sequential Pipeline:** Tasks flow through a fixed sequence of agents. Example: ADK Sequential workflow agent.⁵
 - **Parallel Execution:** Multiple agents work on sub-tasks simultaneously. Example: ADK Parallel workflow agent.⁵
 - **Hierarchical:** Agents organized in a tree structure, with parent agents delegating to children.⁵
 - **Generator-Critic:** One agent proposes solutions, another critiques or refines them.⁸⁹
 - **Challenges:** Increased orchestration complexity, potential for infinite loops if not constrained, difficulty in tracing and debugging interactions across multiple agents.⁴⁸ Requires careful design of communication protocols and state-sharing mechanisms. The emergence of explicit multi-agent support (Agno Teams, LangGraph, OpenAI Handoffs, ADK hierarchies) indicates this is a critical pattern for tackling real-world complexity.¹

C. Agent Execution Loops & Patterns

The agent execution loop defines the iterative process an agent follows to accomplish its task. Different frameworks and patterns implement this loop in various ways.

- **Core Logic:** The fundamental loop involves the LLM reasoning about the current state and goal, deciding on an action (use a tool or respond), executing the

action, observing the result, and repeating until the goal is met or a stopping condition is triggered.²⁷

- **ReAct (Reason+Act):** This pattern makes the reasoning explicit. The LLM generates a "Thought" outlining its reasoning, followed by an "Action" (tool call or final answer). The environment provides an "Observation" (tool result), which feeds into the next Thought step.⁵⁸ LangChain provides specific support for ReAct agents.⁶²
- **Tool Calling:** Modern LLMs often have built-in capabilities to call functions or tools based on their descriptions. Agents using this pattern rely on the model's internal reasoning to select and invoke tools directly, often requiring less explicit prompt engineering for the loop itself.²⁷ LangChain's `create_tool_calling_agent`¹⁶ and OpenAI's function/tool calling¹⁰² exemplify this.
- **SDK-Managed Loops:** Frameworks like the OpenAI Agents SDK and Google ADK provide a Runner class that encapsulates the execution loop.³³ The Runner handles iterating LLM calls, processing tool outputs, managing handoffs (in OpenAI SDK³³), and determining when the final output is reached based on configured criteria (e.g., specific output type or lack of further tool calls).³³
- **LangGraph Loops:** In LangGraph, loops are explicitly defined by the graph structure (edges connecting back to previous nodes), often controlled by conditional edges that check if more work is needed.²²
- **Legacy AgentExecutor (LangChain):** This class provided an earlier implementation of an agent loop.⁶² It included parameters like `max_iterations` and `max_execution_time` to prevent infinite loops⁹³ and options for handling parsing errors.⁶⁶ However, its rigidity led to issues like getting stuck in loops or parsing failures⁶⁶, prompting the recommendation to migrate to LangGraph for better control.²⁷
- **Other Patterns:**
 - **Reflection:** Agents critique and refine their own outputs iteratively.¹¹⁵
 - **Planning:** Agents first generate a plan (sequence of steps) before executing it.¹¹⁵

The choice of execution loop pattern impacts the agent's transparency, controllability, and reliance on the LLM's internal reasoning versus explicit instructions. The move away from black-box executors like the original AgentExecutor towards more transparent and controllable loops (LangGraph, SDK Runners) reflects the need for greater reliability and debuggability in agentic systems.

IV. Building Blocks: A Step-by-Step Guide

This section provides a practical guide to assembling the core components of an AI

agent using concepts and examples drawn from various frameworks.

A. Setting Up the Environment

A robust development environment is the first step.

1. **Python Version:** Ensure a compatible Python version is installed. Requirements vary across frameworks, but recent versions (e.g., 3.9+, 3.10+, 3.12+) are common.¹
2. **Dependency Management:** Use a virtual environment (venv, conda) to isolate project dependencies.¹ Install framework packages and dependencies using tools like pip, uv, or poetry.¹
 - Example (Agno): `uv venv --python 3.12 && source.venv/bin/activate && uv pip install agno openai`¹
 - Example (ADK): `python -m venv.venv && source.venv/bin/activate && pip install google-adk`⁴²
 - Example (OpenAI SDK): `python -m venv env && source env/bin/activate && pip install openai-agents`²⁹
 - Example (LangChain): `pip install langchain langchain-openai tavily-python faiss-cpu`¹⁰⁰ (Dependencies vary based on usage)
3. **API Key Management:** Securely manage API keys for LLM providers and tools. Use environment variables or .env files.¹ Load keys using libraries like python-dotenv³⁵ or access them via `os.getenv()`.²⁹ Avoid hardcoding keys in source code.³¹
 - Example: `export OPENAI_API_KEY='sk-...'`¹
 - Example: Create .env file with `OPENAI_API_KEY=your-key`³¹
4. **Project Structure:** Organize code logically. Google ADK suggests a structure with `__init__.py`, `agent.py`, and `.env` within an agent module folder.⁴² Agno provides cookbook examples with clear file structures.⁷⁶

B. Defining the Agent Core

Instantiate the agent class provided by the framework and configure its core attributes.

1. **Instantiate Agent Class:**
 - Agno: `agent = agno.agent.Agent(...)`¹
 - OpenAI SDK: `agent = agents.Agent(...)`²⁸
 - Google ADK: `agent = google.adk.agents.Agent(...)`⁷⁷
 - LangChain: Define agent logic using LCEL or LangGraph, often involving `create_react_agent` or `create_tool_calling_agent` which return a Runnable.⁵⁸
2. **Configure Model:** Select the LLM and pass its configuration to the agent's model

parameter.

- Example (Agno): `model=OpenAIChat(id="gpt-4o")`¹ or `model=Groq(id="llama-3.1-70b-versatile", api_key=GROQ_API_KEY)`⁵²
 - Example (OpenAI SDK): `model="gpt-4o-mini"`²⁹
 - Example (ADK): `model="gemini-1.5-flash"`⁷⁷
3. **Craft Instructions:** Define the agent's behavior using parameters like instructions, description, goal, role, or system prompts within prompt templates.⁵⁰ Ensure clarity and specificity.²
- Example (Agno): `description="You are a helpful assistant.", instructions=`⁵⁷
 - Example (OpenAI SDK): `instructions="You are a support agent who can submit refunds [...]"`²⁸

C. Integrating Tools

Equip the agent with necessary tools.

1. **Use Pre-built Tools:** Import the tool class, instantiate it (potentially with configuration), and add the instance to the agent's tools list.
 - Example (Agno): `tools=`¹
 - Example (LangChain): `tools =`⁶²
 - Example (OpenAI SDK): `tools=`²⁸
2. **Create Custom Tools:**
 - Define a Python function with clear docstrings describing its purpose, parameters, and return value.
 - Use framework mechanisms to register it: OpenAI SDK's `@function_tool` decorator²⁸; ADK function tools⁷⁷; LangChain tool decorator¹⁴; Agno allows passing callables directly in the tools list.⁵⁷
 - Define input schemas using Pydantic/Zod if needed for structured inputs.⁵⁰
 - Add the custom tool function/object to the agent's tools list.
3. **Configure Tool Usage:** Options like Agno's `show_tool_calls=True` can aid debugging by making tool interactions visible.⁵⁰

D. Implementing Memory

Integrate memory for context persistence.

1. **Choose Memory Type:** Select based on application needs: simple buffer, summary, token limit, vector retrieval, entity tracking, or framework-specific options like ADK's Session/State/Memory.⁷⁸ See Table 3 below for a comparison.
2. **Configure:** Instantiate the chosen memory/storage class.
 - Example (Agno): `storage=SqliteAgentStorage(db_file="./data/db.sqlite", table_name="agent_sessions")`⁵⁴

- Example (LangChain): memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True) ⁸²
- Example (ADK): session_service = InMemorySessionService() ⁸⁷
- 3. **Integrate:** Pass the configured object to the appropriate parameter in the agent or runner initialization.
 - Agno: storage=agent_storage ⁵⁷
 - LangChain: memory=memory in AgentExecutor or chain ⁹²
 - ADK: session_service=session_service in Runner ⁸⁷
- 4. **Enable History:** Set flags if needed to ensure memory content is used in prompts (e.g., Agno add_history_to_messages=True, read_chat_history=True ⁵⁵).

Table 3: Memory Management Options Comparison

Memory Type	Description	Key Frameworks /Implement ations	Pros	Cons	Use Cases
Buffer	Stores entire raw conversation history.	LangChain Conversation BufferMemor y ⁷⁸ , Agno Chat History ⁸	Simple, Complete history.	Can exceed LLM context limits quickly, Inefficient for long conversations.	Short conversations, Debugging.
Windowed Buffer	Stores only the last 'k' interactions.	LangChain Conversation BufferWindowMemory ⁷⁸	Manages context length, Simple.	Loses older context abruptly.	Maintaining recent context, Fixed-size memory constraints.
Token Buffer	Stores recent messages up to a token limit, dropping oldest when	LangChain Conversation TokenBuffer Memory ⁷⁸	Precisely controls token usage, Balances history and limits.	Can still lose important older context.	Strict token limit management , Balancing context length and history.

	full.				
Summary	Uses an LLM to progressively summarize the conversation.	LangChain Conversation SummaryMemory ⁷⁸ , Agno Session Summaries ⁸	Handles very long conversations, Keeps context concise.	Summarization is lossy, Adds LLM call latency/cost, Summary quality varies.	Very long conversations, Extracting key points over time.
Summary Buffer	Combines buffer (for recent messages) and summary (for older messages).	LangChain Conversation SummaryBufferMemory ⁷⁸	Balances recent detail with long-term summary, Efficient context use.	More complex than simple buffer or summary.	Long conversations needing both recent detail and overall context.
Vector Store	Stores conversation turns as embeddings, retrieves relevant turns via search.	LangChain VectorStoreRetrieverMemory ⁷⁸	Retrieves semantically relevant context, Handles long/disjointed convos.	Adds embedding/search overhead, Retrieval quality depends on query/data.	Retrieving specific past information based on current query relevance.
Entity	Extracts and tracks information about key entities mentioned.	LangChain Conversation EntityMemory ⁷⁸	Provides structured knowledge about key topics, Good for entity-centric tasks.	Entity extraction can be imperfect, May miss non-entity context.	Tracking details about specific people, places, products over time.
ADK Session/State	Session holds current interaction events, State holds temporary	Google ADK ⁴¹	Clear separation of concerns (current vs. long-term).	InMemorySessionService is not persistent.	Managing context within a single interaction thread.

	session data.				
ADK MemoryService	Manages searchable, potentially persistent, cross-session knowledge.	Google ADK (InMemory, Vertex AI RAG backends) ⁴¹	Enables long-term recall, Semantic search (with RAG), Scalable (Vertex).	Requires setup (especially Vertex AI RAG), Search adds latency.	Recalling facts from past interactions, Accessing external knowledge bases.
Agno User Memories	Persistent key-value store associated with a user ID.	Agno (SqliteAgent Storage, etc.) ⁸	Persists user preferences/facts across sessions.	Simple key-value structure.	Remembering user preferences, personalization.
LangGraph State	Explicit state object managed within the graph execution.	LangGraph ²⁰	Highly flexible, Explicit control, Supports persistence.	Requires graph definition.	Complex stateful workflows, Multi-agent coordination.

E. Adding Knowledge (RAG)

Integrate external knowledge using RAG.

1. **Setup Vector Store & Embedder:** Configure connection details (URI, keys) and choose embedding models as described previously.⁵⁴
2. **Define Knowledge Source:** Use document loaders to specify the data source (URLs, PDF paths, database connections).¹
3. **Load/Index Data:** Execute the loading process, which typically involves fetching, chunking, embedding, and indexing the data into the vector store. This might be a one-time setup or require periodic updates.¹
4. **Integrate with Agent:** Connect the configured knowledge base to the agent.
 - Agno: Pass AgentKnowledge object to knowledge parameter, set search_knowledge=True.⁵⁰
 - LangChain: Use RAG-specific chains or configure a retriever tool.
 - ADK: Configure MemoryService with Vertex AI RAG backend.⁸⁶
 - OpenAI: Use the file_search tool after uploading files.²⁸

F. Enabling Reasoning

Configure the agent's reasoning approach.

1. **Implicit:** Rely on the LLM and prompt design.
2. **Explicit (CoT/ReAct):** Use specific prompt templates or agent types designed for step-by-step reasoning.⁵⁸
3. **Dedicated Components:** Utilize reasoning-specific tools (Agno ReasoningTools¹) or configure separate reasoning models/agents if the framework supports it (Agno reasoning, reasoning_model⁵⁷).

G. Handling Multi-modality

Extend agents to process and generate non-text data.

1. **Model Selection:** Choose an LLM explicitly supporting multi-modal inputs (e.g., GPT-4o/Vision, Claude 3, Gemini Pro Vision, Agno examples with Pixtral, Aya Vision).¹
2. **Input Handling:** Use framework mechanisms to pass image, audio, or video data.
 - Agno: images parameter in run/print_response accepts file paths, URLs, or Image objects.⁵⁰
 - Other Frameworks: May require encoding (e.g., Base64¹²⁹) or specific input formats.
3. **Output Handling:** If the agent generates media (e.g., using DALL-E via Agno DalleTools¹²⁸), process the output accordingly (e.g., download generated image from URL¹²⁸). Agno cookbook includes examples for video generation and audio I/O.⁷⁶

H. Running the Agent

Execute the configured agent.

1. **Invocation:** Call the appropriate method provided by the framework's agent or runner class.
 - Agno: agent.run(...) or agent.print_response(...)⁵⁰
 - OpenAI SDK: Runner.run_sync(...) or Runner.run(...)²⁸
 - LangChain: agent_executor.invoke(...)⁵⁸
 - ADK: app.query(...) or runner.run_async(...)⁷⁷
2. **Input/Output:** Provide the user query and any necessary context (e.g., session ID, user ID). Process the returned response, which could be text, structured data (JSON), or media.
3. **Streaming:** Utilize streaming capabilities (stream=True¹, run_streamed²⁹) for better perceived performance, especially for long generations or complex chains.

Handle streamed chunks appropriately in the application logic.

Building an agent involves carefully selecting and configuring these blocks. The specific implementation details differ across frameworks, necessitating a good understanding of the chosen framework's API and conventions. Effective tool integration and sophisticated memory management are often key differentiators for capable agents. The rise of multi-modal models and Agentic RAG further expands the possibilities and complexity.

Table 2: Core Framework Components & Functions Mapping

Component	Key Function/Purpose	Agno Implementation	LangChain Implementation	OpenAI SDK Implementation	Google ADK Implementation
Agent Definition	Encapsulates LLM, instructions, tools, memory, etc.	agno.agent. Agent class ⁵⁴	LCEL/LangGraph composition, Agent types	agents.Agent class ²⁸	google.adk.agents.Agent class ⁷⁷
Model Integration	Connects to and configures the LLM	model param (e.g., OpenAIChat) ⁵⁰	ChatModel/LLM interfaces ⁵³	model param (str or Model obj) ³³	model param, LiteLlm wrapper ⁷⁷
Instructions	Guides agent behavior	instructions, description params ⁵⁷	ChatPrompt Template, System Messages ⁵³	instructions param ³³	instruction param ⁴¹
Tool Integration	Provides external capabilities	tools list param, Toolkits ⁵⁷	BaseTool, tool decorator ¹⁴	tools list, @function_tool ²⁸	tools list, Function/OpenAPI tools ⁵
Memory	Stores conversational context/state	Memory/AgentStorage (Sqlite...) ⁵⁷	BaseChatMemory subclasses ⁷⁸	Implicit in loop/context	Session, State, MemoryService ⁹⁰
Knowledge	Accesses	AgentKnowle	Retrievers,	file_search	MemoryServi

(RAG)	external information	dge (UrlKnownled ge...) ⁵⁷	Vector Stores, RAG Chains ⁵³	tool ²⁸	ce (Vertex AI RAG) ⁸⁶
Reasoning	Enables step-by-step thinking, planning	ReasoningTools, reasoning param ⁵⁷	ReAct Agents, CoT Prompting ⁹⁴	Implicit in LLM, Prompting	Implicit in LLM, Prompting
Orchestration/Loop	Manages the agent's execution flow	agent.run(), Teams (route...) ¹	AgentExecutor (legacy), LangGraph ²²	Runner, Handoffs ³³	Runner, Workflow Agents ⁵
Multi-modality	Handles non-text inputs/outputs	Native support (images param) ⁷⁰	Emerging support via models/LCEL	Depends on model capabilities	Depends on model capabilities

V. Advanced Features & Considerations

Beyond the core building blocks, developing robust and production-ready agent frameworks requires addressing several advanced aspects, including multi-agent orchestration, performance, observability, security, and error handling.

A. Multi-Agent Orchestration in Practice

As tasks become too complex for a single agent, decomposing them across multiple specialized agents becomes necessary.⁶¹ Effective orchestration is key.

- **Implementation:** Frameworks provide different mechanisms:
 - **Agno Teams:** Offers route (sequential/conditional), collaborate (shared work), and coordinate (central manager) modes for defining team interactions.¹ Example: A coordinating agent managing a Web Search Agent and a Finance Agent.¹
 - **LangGraph:** Naturally suited for multi-agent systems through its graph structure. Nodes can represent individual agents, and edges define the flow of control and information between them.²² State management allows agents to share context.²⁰
 - **OpenAI SDK Handoffs:** A specific tool-like mechanism (transfer_to_<agent_name>) allowing one agent to delegate the task to another predefined agent.²⁸ Example: A Triage Agent handing off to

specialized support agents.²⁸

- **Google ADK:** Supports hierarchical agent structures and LLM-driven delegation based on agent descriptions.⁵ Workflow agents (Sequential, Parallel) provide structured orchestration.⁵ Supports A2A protocol for interoperability.⁸⁸
- **State/Context Sharing:** Critical for collaboration. LangGraph uses its shared state object.²⁰ ADK uses Session State for temporary data within an interaction managed by a SessionService.⁴¹ Custom mechanisms might be needed depending on the framework and collaboration pattern.
- **Challenges:** Debugging interactions between multiple autonomous agents can be significantly more complex than single-agent systems. Ensuring clear communication protocols and avoiding infinite loops or deadlocks requires careful design and robust orchestration logic.⁴⁹

B. Performance Optimization

Agent performance is crucial for usability and cost-effectiveness. While LLM inference is often the bottleneck, framework overhead and execution strategies play a significant role.

- **Latency:** Minimize time-to-response. Key factors include:
 - *Framework Overhead:* Agent instantiation time and per-step processing. Lightweight frameworks like Agno aim to minimize this, claiming significantly faster instantiation (~2-3μs) compared to alternatives like LangGraph (~20ms).¹
 - *LLM Inference:* Choose faster models or providers (e.g., Groq⁵¹) where appropriate.
 - *Tool Execution:* Optimize tool performance, especially for I/O-bound operations.
- **Memory Footprint:** Reduce RAM usage, important for scaling and cost. Agno also claims a much smaller memory footprint (~3.75-5KiB) per agent compared to LangGraph (~137KiB).¹ Efficient memory management within the agent (e.g., using summaries instead of full buffers) is also vital.
- **Asynchronous Operations:** Leverage async/await for non-blocking I/O (LLM calls, tool API requests, database interactions) to improve concurrency and throughput.¹ Most modern frameworks offer async support (LCEL, Agno, OpenAI SDK, ADK).
- **Streaming:** Stream responses token-by-token from the LLM and intermediate steps back to the user interface. This dramatically improves perceived performance by showing activity immediately.²⁵ Frameworks like LCEL, Agno,

OpenAI SDK, and ADK provide streaming capabilities.⁵⁰ Implementing streaming effectively requires handling partial outputs and updates in the application logic.¹³²

- **Parallel Execution:** Run independent tasks concurrently. LCEL's RunnableParallel automatically parallelizes steps where possible.¹¹⁰ ADK offers Parallel workflow agents.⁵ This can reduce overall latency for tasks involving multiple tool calls or retrievals.
- **Caching:** Store results of expensive operations (LLM calls, tool results) to avoid re-computation.¹³³

Optimizing performance requires considering the entire execution path, from framework overhead to LLM choice and tool efficiency, leveraging techniques like async programming, streaming, and parallelization.

C. Observability and Debugging

Given the non-deterministic nature of LLMs and the complexity of agentic workflows, robust observability and debugging tools are essential.¹⁴

- **Importance:** Understanding *why* an agent made a certain decision, tracing the sequence of tool calls and reasoning steps, and identifying bottlenecks or errors is critical for development and production monitoring.
- **Tracing Platforms & Tools:**
 - **LangSmith:** Tightly integrated with LangChain and LangGraph, providing detailed traces of runs, including LLM inputs/outputs, tool calls, intermediate steps, latency, and token counts. Supports evaluation and monitoring.¹³
 - **OpenAI SDK Tracing:** Built-in tracing capabilities, visualizing agent runs, tool calls, and handoffs. Extensible to various external destinations (Logfire, AgentOps, etc.).²⁸
 - **Agno Monitoring:** Provides a dashboard (agno.com) for real-time monitoring of agent sessions and performance.¹
 - **Langtrace:** An integration specifically mentioned for Agno, automatically tracing agent runs, tool calls, and memory operations.⁷⁴
 - **OpenTelemetry (OTel):** An open standard for observability. Frameworks are increasingly adding OTel support, allowing integration with various backends (Jaeger, Honeycomb, Langfuse, etc.).¹³⁶ LangSmith now offers end-to-end OTel support.¹³⁶
- **Key Data:** Traces should capture inputs, outputs, errors, latency, and metadata for each step (LLM calls, tool executions, memory access, agent transitions/handoffs).³³
- **Debugging Techniques:** Analyze traces visually.²⁸ Enable verbose logging or

specific flags (e.g., `Agno show_tool_calls`⁵⁷, `debug_mode`¹¹). Use interactive development UIs provided by frameworks (ADK adk web⁴³, Agno Playground/Agent UI³) for step-by-step inspection and testing.

The complexity of agent interactions necessitates dedicated observability solutions beyond standard application logging. Platforms like LangSmith or adherence to standards like OpenTelemetry are becoming crucial for managing these systems effectively.

D. Security and Guardrails

Autonomous agents interacting with external systems and user data introduce security considerations.

- **Input Validation:** Protect against malicious inputs, prompt injection, or unintended queries. Guardrails (like in OpenAI SDK²⁸) can perform checks before the agent processes input.
- **Output Validation:** Ensure agent responses are safe, non-toxic, adhere to policies, and meet format requirements. Guardrails can check outputs.²⁸ Using structured output schemas (Pydantic/Zod) helps enforce format.¹
- **Tool Security:** Agents calling tools, especially those executing code (PythonREPLTool⁵⁸, ADK Code Executor⁷⁵) or modifying external systems, pose risks. Sandboxing code execution and carefully managing tool permissions are essential. Prevent agents from using tools in harmful ways through robust instructions and potentially guardrails.
- **Credential Security:** Securely manage API keys and other credentials needed by the agent or its tools, avoiding hardcoding.¹⁸
- **Data Privacy:** Be mindful of sensitive data in prompts or logs. Frameworks or observability tools may offer mechanisms to prevent logging sensitive information.²⁸
- **Responsible AI:** Follow best practices for building trustworthy AI. Google ADK includes a guide on Responsible Agents.⁵

As agents gain autonomy, implementing safety mechanisms like guardrails and secure tool handling becomes paramount.

E. Error Handling and Resilience

Agent workflows can fail due to issues with LLMs, tools, or network connectivity. Building resilient systems requires robust error handling.

- **Tool Failures:** Tools might fail due to network errors, invalid inputs, API changes,

or internal exceptions. The agent framework needs mechanisms to catch these errors and allow the agent to react, perhaps by trying a different tool, asking for clarification, or reporting the failure.⁴⁸ LangChain's AgentExecutor had a `handle_parsing_errors` parameter.⁶⁶

- **LLM Failures:** API calls to LLMs can fail due to rate limits, server downtime, or network issues. Responses might also be malformed or unparsable.¹⁴⁹
- **Retries:** Implement automatic retries for transient errors (temporary network glitches, rate limit hiccups). LCEL provides a `with_retry` method allowing configuration of attempts, backoff strategies, and specific exceptions to retry on.¹⁵² Agno has a `retries` parameter in its Agent class.⁸⁴
- **Fallbacks:** Define alternative execution paths or models if a primary component fails persistently. LCEL's `RunnableWithFallbacks` allows specifying a sequence of fallback `Runnables` to try if the main one errors.¹⁴⁹ This could involve switching to a different LLM provider or using a simpler, potentially hardcoded response.

Building fault tolerance through retries and fallbacks is essential for production-grade agent applications operating in unreliable environments.

VI. Evaluation and Deployment

Developing an agent is only part of the lifecycle; rigorous evaluation and effective deployment are critical for success.

A. Agent Evaluation Strategies

Evaluating agent performance is complex due to their non-deterministic nature and multi-step execution. It requires assessing not just the final output but also the process.

- **Importance:** Standard accuracy metrics are often insufficient. Evaluation needs to consider the quality of reasoning, appropriateness of tool usage, efficiency of the execution path, and overall task success.⁵ It's vital for iteration and improvement.¹³⁴
- **Frameworks & Tools:**
 - **Google ADK:** Provides a dedicated evaluation framework.
 - *Interactive UI (adk web):* Allows real-time testing, session inspection, and saving interaction sessions into eval sets.⁴³
 - *Test Files (.test.json):* For unit-testing specific agent-model interactions, focusing on queries, expected tool calls, and reference responses.¹⁴⁷
 - *Evalsets (.evalset.json):* For integration testing complex, multi-turn conversations, capturing full sessions with initial state.¹⁴⁷ Can be

generated via adk web.

- **CLI (adk eval):** Runs evaluations on evalset files programmatically or from the command line, suitable for CI/CD.¹⁴⁷
- **Metrics:** Configurable via test_config.json, including tool_trajectory_avg_score (comparing actual vs. expected tool use) and response_match_score (comparing final response similarity using ROUGE).¹⁴⁷
- **LangSmith:** A comprehensive platform for LLM application evaluation.
 - **Features:** Supports creating datasets from traces, running evaluations (including LLM-as-Judge where an LLM scores the agent's output), collecting human feedback, monitoring metrics over time, and comparing runs.¹³ Supports online evaluation (evaluating production traffic).¹³⁹
- **Metrics:** Beyond framework-specific metrics, common evaluation criteria include task completion rates, accuracy of information retrieved/generated, relevance of responses, adherence to instructions, number of steps/turns, latency, and cost.¹³⁵ Human evaluation often remains the gold standard for assessing nuanced aspects like helpfulness or tone.

Effective evaluation requires defining clear success criteria, creating representative test cases (or leveraging production data via platforms like LangSmith), and using appropriate tools and metrics to measure performance across multiple dimensions.

B. Deployment Options

Once developed and evaluated, agents need to be deployed into accessible environments.

- **Local Execution:** Running agents via Python scripts or CLIs is suitable for development, testing, and simple use cases.¹ ADK provides adk run for CLI interaction.⁴²
- **API Server:** A common approach is to wrap the agent framework in a web server (like FastAPI) and expose it as a REST API. LangServe is specifically designed for deploying LangChain Runnables.¹⁵ ADK examples show wrapping agents with FastAPI for A2A communication.⁸⁸ This allows integration with web/mobile applications or other backend services.
- **Cloud Platforms:** Leveraging cloud providers offers scalability, manageability, and integration with other services.
 - **Google Cloud:**
 - **Vertex AI Agent Engine:** A managed, auto-scaling service specifically for deploying and scaling AI agents, including those built with ADK.⁵
 - **Cloud Run:** A serverless platform for running containerized applications,

suitable for deploying agent APIs.⁵

- **Google Kubernetes Engine (GKE):** Provides container orchestration for complex deployments, though not explicitly detailed for ADK in all snippets.⁵
- **AWS:** LangChain integrates with AWS services.¹⁵⁴ AWS Bedrock offers its own agent capabilities.¹⁵⁶ Agno provides deployment examples for AWS (agent-app-aws, agent-api-aws).¹⁵⁷
- **Containerization:** Packaging the agent framework and its dependencies into Docker containers allows for consistent deployment across different environments (local, cloud, on-premise).⁵
- **Serverless Functions:** For simpler agents or specific tasks within a larger workflow, deploying agent logic as serverless functions might be viable.¹³⁹

The choice of deployment strategy depends on factors like scalability requirements, existing infrastructure, need for managed services, cost considerations, and integration with other cloud services. Managed platforms like Vertex AI Agent Engine aim to simplify the operational burden of running complex agentic systems at scale.

VII. Conclusion & Future Directions

A. Summary of the Building Plan

Building a functional AI agent framework is a multifaceted endeavor that extends far beyond simply wrapping an LLM call. It requires a systematic approach encompassing the definition of core agent components, careful architectural design, robust implementation of advanced features, and rigorous evaluation and deployment strategies.

1. **Core Concepts:** The foundation rests on integrating an LLM (the reasoning engine) with clear Instructions, empowering it with Tools for external interaction, providing Memory for context, enabling access to external Knowledge (often via Agentic RAG), and facilitating complex Reasoning processes.
2. **Architecture:** Key architectural decisions involve choosing a framework philosophy (Lightweight, Composable, Graph-Based, SDK), deciding between single-agent and multi-agent systems, and selecting an appropriate agent execution loop pattern (ReAct, Tool Calling, Managed Loops). Frameworks like Agno, LangChain/LangGraph, OpenAI SDK, and Google ADK offer different trade-offs in performance, flexibility, control, and ecosystem integration.
3. **Building Blocks:** Implementation involves setting up the environment, defining the agent core, integrating tools, memory, knowledge, and reasoning capabilities, potentially handling multi-modality, and finally, running the agent and managing

its input/output, often utilizing streaming.

4. **Advanced Considerations:** Production-readiness demands attention to multi-agent orchestration, performance optimization (latency, memory, async/streaming), comprehensive observability and debugging, security measures (guardrails, tool safety), and error handling (retries, fallbacks).
5. **Lifecycle Management:** The process concludes with thorough evaluation using specialized metrics and tools, followed by deployment to suitable environments (local, API server, cloud platforms).

B. Key Trade-offs

Developers building or choosing an agent framework constantly navigate critical trade-offs:

- **Performance vs. Flexibility/Abstraction:** Highly optimized, lightweight frameworks (like Agno) may offer superior speed and efficiency but potentially less built-in abstraction or a smaller ecosystem compared to more comprehensive, composable frameworks (like LangChain) which might introduce more overhead.
- **Control vs. Ease of Use:** Low-level, code-first frameworks (like ADK or LangGraph) provide fine-grained control over execution but have a steeper learning curve compared to frameworks with higher-level abstractions or pre-built agent types.
- **Ecosystem Integration vs. Agnosticism:** Frameworks tightly integrated with specific platforms (ADK/Google Cloud, OpenAI SDK/OpenAI) offer seamless integration but may limit flexibility, whereas model-agnostic frameworks (Agno, LangChain) provide broader compatibility but might require more configuration effort.
- **Single-Agent Simplicity vs. Multi-Agent Complexity:** While multi-agent systems offer modularity and specialization for complex tasks, they introduce significant orchestration and debugging challenges compared to single-agent designs.

C. Emerging Trends

The field of AI agent frameworks is dynamic, with several trends shaping its future:

- **Enhanced Multi-modality:** Agents are increasingly expected to natively handle and generate not just text, but also images, audio, and video, requiring framework support beyond text processing.¹
- **Sophisticated Multi-Agent Collaboration:** Moving beyond simple delegation towards more complex collaboration patterns, potentially facilitated by

standardization efforts like the Model Context Protocol (MCP) or Agent2Agent (A2A) for interoperability.⁴⁵

- **Improved Reasoning and Planning:** Development of more advanced reasoning techniques (beyond basic ReAct or CoT) and planning capabilities integrated directly into frameworks.
- **Standardized Evaluation:** Emergence of common benchmarks and more robust methodologies for evaluating agent performance across various dimensions (task success, reasoning quality, safety, efficiency).
- **Developer Experience:** Tighter integration with IDEs (e.g., Agno/Serena Cursor integration ¹) and visual development/debugging tools (e.g., LangGraph Studio ²⁵, ADK Web UI ¹⁴⁷).
- **Increased Focus on Safety and Responsibility:** Integration of more sophisticated guardrails, alignment techniques, and tools for building trustworthy and reliable agents.⁵

D. Final Recommendations

Building effective AI agent frameworks requires a thoughtful, iterative approach:

1. **Start Simple, Iterate:** Begin with a clear definition of the core problem and potentially a single-agent solution using a chosen framework. Gradually add complexity – tools, memory, RAG, multi-agent patterns – as needed, evaluating at each stage.
2. **Choose Architecture Wisely:** Select a framework or architectural pattern that aligns with project requirements, team expertise, and performance/scalability needs. Consider the trade-offs between different philosophies (lightweight vs. comprehensive, control vs. abstraction).
3. **Prioritize Observability:** Implement robust tracing and logging from the outset. Understanding agent behavior is critical for debugging and improvement. Leverage platforms like LangSmith or standards like OpenTelemetry.
4. **Evaluate Rigorously:** Define clear success metrics and use framework-specific tools (like ADK evaluation) or dedicated platforms (like LangSmith) to assess performance comprehensively, including reasoning and tool use, not just final output accuracy. Incorporate human feedback where appropriate.
5. **Design for Resilience and Safety:** Implement error handling (retries, fallbacks) and security measures (input/output validation, tool sandboxing) proactively.

The development of AI agents and the frameworks that support them is rapidly advancing. By understanding the core concepts, architectural patterns, and practical considerations outlined in this blueprint, developers can navigate this complex

landscape and build increasingly capable, reliable, and functional agentic systems.

Works cited

1. Agno is a lightweight library for building Agents with memory, knowledge, tools and reasoning. - GitHub, accessed April 21, 2025, <https://github.com/agno-agi/agno>
2. Building Effective AI Agents - Anthropic, accessed April 21, 2025, <https://www.anthropic.com/research/building-effective-agents>
3. xAI Grok + Cursor + Agno: Build a Multi-Agent AI App in Python - GetStream.io, accessed April 21, 2025, <https://getstream.io/blog/xai-python-multi-agent/>
4. Building a Multi-Agent Financial News System with Agno and Agent UI - Marketcalls, accessed April 21, 2025, <https://www.marketcalls.in/python/building-a-multi-agent-financial-news-system-with-agno-and-agent-ui.html>
5. Agent Development Kit - Google, accessed April 21, 2025, <https://google.github.io/adk-docs/>
6. A Guide to Large Language Model Abstractions - Two Sigma, accessed April 21, 2025, <https://www.twosigma.com/articles/a-guide-to-large-language-model-abstractions/>
7. agno · PyPI, accessed April 21, 2025, <https://pypi.org/project/agno/1.0.5/>
8. From Zero to Agent Hero: Getting Started with Agno Agents, uv, and a Dash of RAG Magic, accessed April 21, 2025, <https://www.bitdoze.com/agno-get-start/>
9. Unleashing the Power of Agno: Building Multi-Modal Agents with a Lightweight Python Library - RPABOTS.WORLD, accessed April 21, 2025, <https://rpabotsworld.com/agno-building-multi-modal-agents-with-a-lightweight-python-library/>
10. Agno Framework: A Lightweight Library for Building Multimodal Agents - Analytics Vidhya, accessed April 21, 2025, <https://www.analyticsvidhya.com/blog/2025/03/agno-framework/>
11. Top 5 Open Source Frameworks for building AI Agents - Athina AI Hub, accessed April 21, 2025, <https://hub.athina.ai/top-5-open-source-frameworks-for-building-ai-agents-with-examples/>
12. Introduction | 🦜 LangChain, accessed April 21, 2025, <https://python.langchain.com/docs/introduction/>
13. langchain-ai/langchain: Build context-aware reasoning applications - GitHub, accessed April 21, 2025, <https://github.com/langchain-ai/langchain>
14. Towards LangChain 0.1: LangChain-Core and LangChain-Community - LangChain Blog, accessed April 21, 2025, <https://blog.langchain.dev/the-new-langchain-architecture-langchain-core-v0-1-langchain-community-and-a-path-to-langchain-v0-1/>
15. Architecture | 🦜 LangChain, accessed April 21, 2025, <https://python.langchain.com/docs/concepts/architecture/>

16. Overview - LangChain, accessed April 21, 2025, https://python.langchain.com/docs/versions/v0_2/overview/
17. AutoGen vs LangChain: Comparison for LLM Applications - PromptLayer, accessed April 21, 2025, <https://blog.promptlayer.com/autogen-vs-langchain/>
18. LangChain Academy - GitHub, accessed April 21, 2025, <https://github.com/langchain-ai/langchain-academy>
19. About using tools in langgraph : r/LangChain - Reddit, accessed April 21, 2025, https://www.reddit.com/r/LangChain/comments/1fsx8at/about_using_tools_in_langgraph/
20. How to manage agent steps - GitHub Pages, accessed April 21, 2025, <https://langchain-ai.github.io/langgraphjs/how-tos/managing-agent-steps/>
21. LangGraph Tutorial: Building LLM Agents with LangChain's Agent Framework - Zep, accessed April 21, 2025, <https://www.getzep.com/ai-agents/langgraph-tutorial>
22. LangGraph - LangChain Blog, accessed April 21, 2025, <https://blog.langchain.dev/langgraph/>
23. Introduction to LangGraph - LangChain Academy, accessed April 21, 2025, <https://academy.langchain.com/courses/intro-to-langgraph>
24. Complete Guide to Building LangChain Agents with the LangGraph Framework - Zep, accessed April 21, 2025, <https://www.getzep.com/ai-agents/langchain-agents-langgraph>
25. LangGraph - LangChain, accessed April 21, 2025, <https://www.langchain.com/langgraph>
26. learn to create your first AI agent easily : r/LangChain - Reddit, accessed April 21, 2025, https://www.reddit.com/r/LangChain/comments/1im696r/learn_to_create_your_first_ai_agent_easily/
27. Agents | 🦜 LangChain, accessed April 21, 2025, <https://python.langchain.com/docs/concepts/agents/>
28. New tools for building agents | OpenAI, accessed April 21, 2025, <https://openai.com/index/new-tools-for-building-agents/>
29. cookbook/gen-ai/openai/agents-sdk-intro.ipynb at main - GitHub, accessed April 21, 2025, <https://github.com/aurelio-labs/cookbook/blob/main/gen-ai/openai/agents-sdk-intro.ipynb>
30. A practical guide to building agents | OpenAI, accessed April 21, 2025, <https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf>
31. How to Use the OpenAI Agents SDK ? - Apidog, accessed April 21, 2025, <https://apidog.com/blog/how-to-use-openai-agents-sdk/>
32. lastmile-ai/openai-agents-mcp: An MCP extension package for OpenAI Agents SDK - GitHub, accessed April 21, 2025, <https://github.com/lastmile-ai/openai-agents-mcp>
33. openai/openai-agents-python: A lightweight, powerful framework for multi-agent workflows, accessed April 21, 2025,

- <https://github.com/openai/openai-agents-python>
34. Building Agentic AI Applications using OpenAI Agents SDK - Association of Data Scientists, accessed April 21, 2025,
<https://adasci.org/building-agentic-ai-applications-using-openai-agents-sdk/>
 35. OpenAI Agents SDK Tutorial: Building AI Systems That Take Action | DataCamp, accessed April 21, 2025,
<https://www.datacamp.com/tutorial/openai-agents-sdk-tutorial>
 36. OpenAI Agents SDK — Getting Started - GetStream.io, accessed April 21, 2025,
<https://getstream.io/blog/openai-agents-sdk/>
 37. OpenAI Open-Sources Agents SDK: Building Agentic AI Applications - Learn Prompting, accessed April 21, 2025,
<https://learnprompting.org/blog/openai-agents-sdk>
 38. A Deep Dive Into The OpenAI Agents SDK - Sid Bharath, accessed April 21, 2025,
<https://www.siddharthbharath.com/openai-agents-sdk/>
 39. Deep-Insight-Labs/awesome-ai-agents - GitHub, accessed April 21, 2025,
<https://github.com/Deep-Insight-Labs/awesome-ai-agents>
 40. Top OpenAI Agents SDK Alternatives in 2025, accessed April 21, 2025,
<https://slashdot.org/software/p/OpenAI-Agents-SDK/alternatives>
 41. How to Build Your First Agent with Google Agent Development Kit (ADK) - BitDoze, accessed April 21, 2025, <https://www.bitdoze.com/google-adk-start/>
 42. Quickstart: Build an agent with the Agent Development Kit | Generative AI on Vertex AI, accessed April 21, 2025,
<https://cloud.google.com/vertex-ai/generative-ai/docs/agent-development-kit/quickstart>
 43. Unlock Advanced AI: An Introduction to Google's Agent Development Kit (ADK), accessed April 21, 2025,
<https://wowdata.science/unlock-advanced-ai-an-introduction-to-googles-agent-development-kit-adk/>
 44. google/adk-docs: An open-source, code-first Python toolkit for building, evaluating, and deploying sophisticated AI agents with flexibility and control. - GitHub, accessed April 21, 2025, <https://github.com/google/adk-docs>
 45. Agent Development Kit: Making it easy to build multi-agent applications, accessed April 21, 2025,
<https://developers.googleblog.com/en/agent-development-kit-easy-to-build-multi-agent-applications/>
 46. Agno (Phidata) - Framework AI Agent Builder, accessed April 21, 2025,
<https://bestaiagents.ai/agent/agno>
 47. Top AI agent builders and frameworks for various use cases : r/AI_Agents - Reddit, accessed April 21, 2025,
https://www.reddit.com/r/AI_Agents/comments/1jfk4rz/top_ai_agent_builders_and_frameworks_for_various/
 48. Agent system design patterns - Azure Databricks | Microsoft Learn, accessed April 21, 2025,
<https://learn.microsoft.com/en-us/azure/databricks/generative-ai/guide/agent-system-design-patterns>

49. Agent system design patterns - Databricks Documentation, accessed April 21, 2025,
<https://docs.databricks.com/aws/en/generative-ai/guide/agent-system-design-patterns>
50. agno-agi/phidata - GitHub, accessed April 21, 2025,
<https://github.com/agno-agi/phidata>
51. How To Build Financial Agent with Agno & Groq - Dataaspirant, accessed April 21, 2025, <https://dataaspirant.com/building-financial-agent-agno-groq/>
52. Agno - GroqCloud, accessed April 21, 2025, <https://console.groq.com/docs/agno>
53. Conceptual guide | 🦜 LangChain, accessed April 21, 2025,
<https://python.langchain.com/v0.2/docs/concepts/>
54. agno/cookbook/examples/agents/agno_support_agent.py at main - GitHub, accessed April 21, 2025,
https://github.com/agno-agi/agno/blob/main/cookbook/examples/agents/agno_support_agent.py
55. agno/cookbook/examples/agents/deep_knowledge.py at main - GitHub, accessed April 21, 2025,
https://github.com/agno-agi/agno/blob/main/cookbook/examples/agents/deep_knowledge.py
56. agno/cookbook/examples/agents/media_trend_analysis_agent.py at main - GitHub, accessed April 21, 2025,
https://github.com/agno-agi/agno/blob/main/cookbook/examples/agents/media_trend_analysis_agent.py
57. Agent - Agno, accessed April 21, 2025,
<https://docs.agno.com/reference/agents/agent>
58. Python - LangChain, accessed April 21, 2025,
<https://python.langchain.com/v0.1/docs/integrations/toolkits/python/>
59. agno/cookbook/examples/agents/agno_assist.py at main - GitHub, accessed April 21, 2025,
https://github.com/agno-agi/agno/blob/main/cookbook/examples/agents/agno_assist.py
60. agno/cookbook/examples/agents/legal_consultant.py at main - GitHub, accessed April 21, 2025,
https://github.com/agno-agi/agno/blob/main/cookbook/examples/agents/legal_consultant.py
61. Read This Before Building AI Agents: Lessons From The Trenches - DEV Community, accessed April 21, 2025,
<https://dev.to/isaachagoel/read-this-before-building-ai-agents-lessons-from-the-trenches-333i>
62. ReAct - LangChain, accessed April 21, 2025,
https://python.langchain.com/v0.1/docs/modules/agents/agent_types/react/
63. 11-langchain-expression-language.ipynb - GitHub, accessed April 21, 2025,
<https://github.com/pinecone-io/examples/blob/master/learn/generation/langchain/handbook/11-langchain-expression-language.ipynb>
64. LangChain Expression Language (LCEL), accessed April 21, 2025,

- https://python.langchain.com/v0.1/docs/expression_language/
65. LangChain Agent Executor Deep Dive - Aurelio AI, accessed April 21, 2025, <https://www.aurelio.ai/learn/langchain-agent-executor>
 66. Langchain agent loops over the same tool producing the same result. #20501 - GitHub, accessed April 21, 2025, <https://github.com/langchain-ai/langchain/discussions/20501>
 67. tools — LangChain documentation - LangChain, accessed April 21, 2025, https://python.langchain.com/api_reference/core/tools.html
 68. Introduction - Agno, accessed April 21, 2025, <https://docs.agno.com/tools/introduction>
 69. Building Dissi with Agno and MCP - DEV Community, accessed April 21, 2025, <https://dev.to/komsenapati/building-dissi-with-agno-and-mcp-4044>
 70. agno/cookbook/models/mistral/image_file_input_agent.py at main - GitHub, accessed April 21, 2025, https://github.com/agno-agi/agno/blob/main/cookbook/models/mistral/image_file_input_agent.py
 71. lulu3202/agno_usecase: This repository demonstrates the implementation of a multi-agent system using the AGNO framework, combining web search, GitHub code search, and GIF integration capabilities., accessed April 21, 2025, https://github.com/lulu3202/agno_usecase
 72. Agno - Tavily Docs, accessed April 21, 2025, <https://docs.tavily.com/documentation/integrations/agno>
 73. Build an Agent - LangChain, accessed April 21, 2025, <https://python.langchain.com/docs/tutorials/agents/>
 74. Agno - Langtrace AI Docs, accessed April 21, 2025, <https://docs.langtrace.ai/supported-integrations/llm-frameworks/agno>
 75. Kjdragan/google-adk-tutorial - GitHub, accessed April 21, 2025, <https://github.com/Kjdragan/google-adk-tutorial>
 76. agno/cookbook/getting_started/README.md at main - GitHub, accessed April 21, 2025, https://github.com/agno-agi/agno/blob/main/cookbook/getting_started/README.md
 77. Develop an Agent Development Kit agent | Generative AI on Vertex AI - Google Cloud, accessed April 21, 2025, <https://cloud.google.com/vertex-ai/generative-ai/docs/agent-engine/develop/adk>
 78. memory — LangChain documentation, accessed April 21, 2025, https://python.langchain.com/api_reference/langchain/memory.html
 79. memory — LangChain documentation, accessed April 21, 2025, <https://api.python.langchain.com/en/latest/langchain/memory.html>
 80. memory — LangChain documentation, accessed April 21, 2025, <https://api.python.langchain.com/en/latest/core/memory.html>
 81. accessed January 1, 1970, <https://python.langchain.com/docs/modules/memory/>
 82. ConversationBufferMemory — LangChain documentation, accessed April 21, 2025, https://python.langchain.com/api_reference/langchain/memory/langchain.memor

[y.buffer.ConversationBufferMemory.html](#)

83. Cookbook 03_memories_and_summaries.py example program FAILS with Ollama · Issue #1688 · agno-agi/agno - GitHub, accessed April 21, 2025, <https://github.com/agno-agi/agno/issues/1688>
84. [Bug] Memory leak when making multiple queries one after another · Issue #2632 - GitHub, accessed April 21, 2025, <https://github.com/agno-agi/agno/issues/2632>
85. Working With Agent Memory | Agno AI Agent Tutorial - YouTube, accessed April 21, 2025, <https://www.youtube.com/watch?v=aKCgig9z7JA>
86. Memory - Agent Development Kit - Google, accessed April 21, 2025, <https://google.github.io/adk-docs/sessions/memory/>
87. Build Your First Intelligent Agent Team: A Progressive Weather Bot with ADK - Colab - Google, accessed April 21, 2025, https://colab.research.google.com/github/google/adk-docs/blob/main/examples/python/notebooks/adk_tutorial.ipynb
88. Google's Agent Development Kit (ADK): A Guide With Demo Project - DataCamp, accessed April 21, 2025, <https://www.datacamp.com/tutorial/agent-development-kit-adk>
89. Getting Started with Google ADK: Build Agentic AI Systems with Ease - Sanket Daru, accessed April 21, 2025, <https://sanketdaru.com/blog/google-adk-agentic-ai-framework-getting-started/>
90. Introduction to Conversational Context: Session, State, and Memory - Agent Development Kit - Google, accessed April 21, 2025, <https://google.github.io/adk-docs/sessions/>
91. Re: Google ADK and large datasets, accessed April 21, 2025, <https://www.googlecloudcommunity.com/gc/AI-ML/Google-ADK-and-large-data-sets/m-p/896078>
92. How to create a langchain doc from an str? - Stack Overflow, accessed April 21, 2025, <https://stackoverflow.com/questions/76551067/how-to-create-a-langchain-doc-from-an-str>
93. AgentExecutor — LangChain documentation, accessed April 21, 2025, <https://api.python.langchain.com/en/latest/langchain/agents/langchain.agents.agent.AgentExecutor.html>
94. Using LangChain ReAct Agents to Answer Complex Questions | Airbyte, accessed April 21, 2025, <https://airbyte.com/data-engineering-resources/using-langchain-react-agents>
95. GitHub | 🦉 Langchain, accessed April 21, 2025, https://js.langchain.com/docs/integrations/document_loaders/web_loaders/github/
96. GitHub - LangChain, accessed April 21, 2025, https://python.langchain.com/docs/integrations/document_loaders/github/
97. LangChain documentation, accessed April 21, 2025, https://python.langchain.com/api_reference/core/documents/langchain_core.documents.base.Document.html
98. documents - LangChain, accessed April 21, 2025,

- https://python.langchain.com/api_reference/core/documents.html
99. Agno - Weaviate, accessed April 21, 2025, <https://weaviate.io/developers/integrations/llm-agent-frameworks/agno>
 100. Build an Agent with AgentExecutor (Legacy) - LangChain, accessed April 21, 2025, https://python.langchain.com/docs/how_to/agent_executor/
 101. langchain · PyPI, accessed April 21, 2025, <https://pypi.org/project/langchain/>
 102. Agent — LangChain documentation, accessed April 21, 2025, <https://api.python.langchain.com/en/latest/langchain/agents/langchain.agents.agent.Agent.html>
 103. ReActChain — LangChain documentation, accessed April 21, 2025, https://python.langchain.com/api_reference/langchain/agents/langchain.agents.react.base.ReActChain.html
 104. agents — LangChain documentation, accessed April 21, 2025, https://python.langchain.com/api_reference/langchain/agents.html
 105. ReActChain - Langchain Python API Reference, accessed April 21, 2025, <https://api.python.langchain.com/en/latest/langchain/agents/langchain.agents.react.base.ReActChain.html>
 106. Agni: Our Graph Based Agent Framework in TypeScript - WarpBuild, accessed April 21, 2025, <https://www.warpbuild.com/blog/agni>
 107. What frameworks are you using for building Agents? : r/AI_Agents - Reddit, accessed April 21, 2025, https://www.reddit.com/r/AI_Agents/comments/1k18q51/what_frameworks_are_you_using_for_building_agents/
 108. Guide to LangChain Runnable Architecture - DZone, accessed April 21, 2025, <https://dzone.com/articles/guide-to-langchain-runnable-architecture>
 109. LangChain Expression Language Explained - Pinecone, accessed April 21, 2025, <https://www.pinecone.io/learn/series/langchain/langchain-expression-language/>
 110. LangChain Expression Language (LCEL), accessed April 21, 2025, <https://python.langchain.com/docs/concepts/lcel/>
 111. Efficient LLM Workflows with LangChain Expression Language - Analytics Vidhya, accessed April 21, 2025, <https://www.analyticsvidhya.com/blog/2024/06/langchain-expression-language/>
 112. LangChain Expression Language (LCEL), accessed April 21, 2025, <https://js.langchain.com/docs/concepts/lcel/>
 113. Unleashing the Power of LangChain Expression Language (LCEL): from proof of concept to production - Artefact, accessed April 21, 2025, <https://www.artefact.com/blog/unleashing-the-power-of-langchain-expression-language-lcel-from-proof-of-concept-to-production/>
 114. Handoffs - OpenAI Agents SDK, accessed April 21, 2025, <https://openai.github.io/openai-agents-python/handoffs/>
 115. Top 4 Agentic AI Design Patterns for Architecting AI Systems - Analytics Vidhya, accessed April 21, 2025, <https://www.analyticsvidhya.com/blog/2024/10/agentic-design-patterns/>
 116. Fun with Agents - Simple example of an Agent Hand-off - API - OpenAI

- Developer Forum, accessed April 21, 2025,
<https://community.openai.com/t/fun-with-agents-simple-example-of-an-agent-h-and-off/1142697>
117. agents — LangChain documentation, accessed April 21, 2025,
<https://api.python.langchain.com/en/latest/core/agents.html>
118. accessed January 1, 1970, <https://python.langchain.com/docs/modules/agents/>
119. AgentExecutor — LangChain documentation, accessed April 21, 2025,
https://python.langchain.com/api_reference/langchain/agents/langchain.agents.agent.AgentExecutor.html
120. langchain.agents.agent.AgentExecutor — LangChain 0.2.17, accessed April 21, 2025,
<https://api.python.langchain.com/en/latest/agents/langchain.agents.agent.AgentExecutor.html>
121. Agent Executor stuck in loop · langchain-ai langchain · Discussion #12674 - GitHub, accessed April 21, 2025,
<https://github.com/langchain-ai/langchain/discussions/12674>
122. A2A/samples/python/agents/google_adk/README.md at main - GitHub, accessed April 21, 2025,
https://github.com/google/A2A/blob/main/samples/python/agents/google_adk/README.md
123. A collection of sample agents built with Agent Development (ADK) - GitHub, accessed April 21, 2025, <https://github.com/google/adk-samples>
124. langchain-ai/langchain-google - GitHub, accessed April 21, 2025,
<https://github.com/langchain-ai/langchain-google>
125. AI Observability with LangSmith - Aurelio AI, accessed April 21, 2025,
<https://www.aurelio.ai/learn/langsmith-intro>
126. agno/cookbook/models/cohere/image_agent.py at main - GitHub, accessed April 21, 2025,
https://github.com/agno-agi/agno/blob/main/cookbook/models/cohere/image_agent.py
127. How to send a image to the llama model using Groq - General - Agno, accessed April 21, 2025,
<https://community.agno.com/t/how-to-send-a-image-to-the-llama-model-using-groq/294>
128. agno/cookbook/tools/dalle_tools.py at main - GitHub, accessed April 21, 2025,
https://github.com/agno-agi/agno/blob/main/cookbook/tools/dalle_tools.py
129. How can I send a picture to my AI agent as input? : r/crewai - Reddit, accessed April 21, 2025,
https://www.reddit.com/r/crewai/comments/1jlb3os/how_can_i_send_a_picture_to_my_ai_agent_as_input/
130. Streaming - OpenAI Agents SDK, accessed April 21, 2025,
<https://openai.github.io/openai-agents-python/streaming/>
131. Handoffs - OpenAI Agents SDK, accessed April 21, 2025,
<https://openai.github.io/openai-agents-python/ref/handoffs/>
132. able to use streamingConfigurations in AgentsforBedrockRuntime

- invoke_agent boto3 sdk, accessed April 21, 2025,
<https://repost.aws/questions/QUuzL1XVQ0TVyp5dEPZn5BZw/able-to-use-streamingconfigurations-in-agentsforbedrockruntime-invoke-agent-boto3-sdk>
133. Conceptual guide - LangChain, accessed April 21, 2025,
<https://python.langchain.com/docs/concepts/>
 134. Add observability to your LLM application | 🦜🔧 LangSmith - LangChain, accessed April 21, 2025,
<https://docs.smith.langchain.com/observability/tutorials/observability>
 135. Get started with LangSmith | 🦜🔧 LangSmith, accessed April 21, 2025,
<https://docs.smith.langchain.com/>
 136. Introducing End-to-End OpenTelemetry Support in LangSmith - LangChain Blog, accessed April 21, 2025,
<https://blog.langchain.dev/end-to-end-opentelemetry-langsmith/>
 137. Observability Quick Start - 🦜🔧 LangSmith - LangChain, accessed April 21, 2025, <https://docs.smith.langchain.com/observability>
 138. Observability Integrations: LangSmith - AI SDK, accessed April 21, 2025,
<https://sdk.vercel.ai/providers/observability/langsmith>
 139. Observability how-to guides | 🦜🔧 LangSmith - LangChain, accessed April 21, 2025, https://docs.smith.langchain.com/observability/how_to_guides
 140. LangSmith - LangChain, accessed April 21, 2025,
<https://www.langchain.com/langsmith>
 141. A Quick Guide for OpenTelemetry Python Instrumentation - Last9, accessed April 21, 2025, <https://last9.io/blog/opentelemetry-python-instrumentation/>
 142. Use OpenTelemetry to instrument a Python application and report trace data - Alibaba Cloud, accessed April 21, 2025,
<https://www.alibabacloud.com/help/en/opentelemetry/user-guide/use-managed-service-for-opentelemetry-to-submit-the-trace-data-of-python-applications>
 143. Send Data with the OpenTelemetry Python SDK - Honeycomb.io Documentation, accessed April 21, 2025,
<https://docs.honeycomb.io/send-data/python/opentelemetry-sdk/>
 144. Python | OpenTelemetry, accessed April 21, 2025,
<https://opentelemetry.io/docs/languages/python/>
 145. Instrumentation | OpenTelemetry, accessed April 21, 2025,
<https://opentelemetry.io/docs/languages/python/instrumentation/>
 146. Example: Using OpenTelemetry SDK with Langfuse OTel API, accessed April 21, 2025, <https://langfuse.com/docs/opentelemetry/example-python-sdk>
 147. Why Evaluate Agents - Agent Development Kit - Google, accessed April 21, 2025, <https://google.github.io/adk-docs/evaluate/>
 148. agno-agi/agent-ui: A modern chat interface for AI agents built with Next.js, Tailwind CSS, and TypeScript. - GitHub, accessed April 21, 2025,
<https://github.com/agno-agi/agent-ui>
 149. Fallbacks - LangChain.js, accessed April 21, 2025,
<https://js.langchain.com/v0.1/docs/guides/fallbacks/>
 150. Fallbacks - LangChain.js, accessed April 21, 2025,
https://js.langchain.com/docs/how_to/fallbacks/

151. Fallbacks | 🦉 LangChain, accessed April 21, 2025, <https://python.langchain.com/v0.1/docs/guides/productionization/fallbacks/>
152. RunnableWithFallbacks — LangChain documentation, accessed April 21, 2025, https://python.langchain.com/api_reference/core/runnables/langchain_core.runnables.fallbacks.RunnableWithFallbacks.html
153. Deploying Your Agent - Agent Development Kit - Google, accessed April 21, 2025, <https://google.github.io/adk-docs/deploy/>
154. LangChain Python API Reference, accessed April 21, 2025, https://python.langchain.com/v0.2/api_reference/
155. LangChain Python API Reference, accessed April 21, 2025, https://python.langchain.com/api_reference/
156. Amazon Bedrock Agents Runtime examples using SDK for Python (Boto3), accessed April 21, 2025, https://docs.aws.amazon.com/code-library/latest/ug/python_3_bedrock-agent-runtime_code_examples.html
157. Agno - GitHub, accessed April 21, 2025, <https://github.com/agno-agi>
158. oraios/serena: a powerful coding agent toolkit providing semantic retrieval and editing capabilities (MCP server & Agno integration) - GitHub, accessed April 21, 2025, <https://github.com/oraios/serena>